

DATA PREPROCESSING AND DATASET PREPARATION FOR MACHINE LEARNING

Assignment Report – MSc Data Science & Artificial Intelligence

Student Name	Arpit Dahyabhai Parmar
Roll Number	1200324
Course	Data Preprocessing
Programme	MSc Data Science & Artificial Intelligence
Assignment Title	Data Preprocessing & Dataset Preparation for Machine Learning

1. Dataset Description

1.1 Overview

For this assignment, I have worked on a real-world Laptop Price Dataset, which is a raw, uncleaned collection of laptop product listings scraped from an e-commerce platform. The raw dataset contained 1,303 records and 12 features, covering a broad range of laptop specifications from multiple well-known manufacturers. The primary objective of this dataset is to enable price prediction based on hardware configuration and brand, making it a classic regression problem in machine learning context.

1.2 Column Descriptions

The following table summarize all original 12 columns in the dataset, along with their data types and a brief description of what each feature represents:

Feature	Attr. Type	Data Type	Description
Unnamed: 0	Numeric (Discrete)	Integer	Original row index from the source CSV. Carries no predictive information; dropped during preprocessing.
Company	Nominal	String	Laptop manufacturer brand (e.g., Dell, Apple, HP, Lenovo).
TypeName	Nominal	String	Category of laptop such as Notebook, Gaming, Ultrabook, Workstation, etc.
Inches	Numeric (Continuous)	Float	Diagonal screen size. Some values were in centimeters and had to be converted to inches for consistency.
ScreenResolution	Nominal	String	Raw text combining display resolution (e.g., 1920x1080) and panel technology (e.g., IPS, 4K Ultra HD, Touchscreen).
Cpu	Nominal	String	Full CPU description including brand, model name, and clock speed (e.g., Intel Core i5 2.3GHz).
Ram	Numeric (Discrete)	String	Amount of RAM (e.g., '8GB'). Stored as string with unit suffix.
Memory	Nominal	String	Combined storage field, describing type and capacity (e.g., '256GB SSD', '1TB HDD').

Feature	Attr. Type	Data Type	Description
Gpu	Nominal	String	Graphics Processing Unit full specification (e.g., Nvidia GeForce GTX 1050).
OpSys	Nominal	String	Pre-installed operating system (e.g., Windows 10, macOS, Linux, No OS).
Weight	Numeric (Continuous)	String	Physical weight of the laptop with 'kg' suffix.
Price	Numeric (Continuous)	Float	Sale price of the laptop in local currency. This is the target variable for prediction.

1.3 Dataset Characteristics

From a structural standpoint, the original dataset was mixed in nature — containing both numerical and categorical features.

2. Data Quality Analysis

2.1 Missing Value Analysis

Upon initial inspection using `df.isnull().sum()`, I found that every single column in the dataset had exactly 30 missing values. At first glance, this seemed like a coincidence, but closer inspection revealed that these 30 nulls all belongs to the same 30 rows. In other words, these 30 records were completely empty — every single field across all 12 columns was null. This pattern strongly suggests a systemic data entry failure rather than random or occasional missingness.

Imputation techniques like mean imputation, median, or even more advanced methods like MICE would be totally inappropriate here since there was absolutely no auxiliary information available in those rows to guide the imputation — even the target variable (Price) was missing. Imputing the dependent variable Price alongside all the predictors would have introduced significant synthetic noise and likely biased the models coefficients. Hence, listwise deletion (case deletion) was the only logically valid choice here. After removing these ghost records, the dataset was reduced to 1,273 usable observations.

2.2 Duplicate Records

After handling the missing rows, I ran a full duplicate check using `df.duplicated()` across all features. The result came back as zero duplicates, which was actually a positive sign. Each of the 1,273 remaining rows represented a unique combination of hardware specs and pricing. This eliminates the concern of certain product listings being artificially over-represented in training data, which would have skewed the model towards those configurations.

2.3 Outlier & Noise Analysis

The Price distribution was found to be heavily right-skewed (positively skewed), which immediately ruled out using Z-score normalisation for outlier detection since that technique assumes a roughly Gaussian / symmetric distribution. Instead, I applied the Interquartile Range (IQR) method, which is a non-parametric approach well-suited for skewed distributions.

Using Tukey's fences, the statistical boundaries were computed as:

- Q1 (25th Percentile) = ₹31,914.72
- Q3 (75th Percentile) = ₹79,333.39

- $IQR = ₹47,418.67$
- Upper Bound ($Q3 + 1.5 \times IQR$) = ₹150,461.39

A total of **28 observations** exceeded the upper bound. However, after carefully reviewing these records individually, it became very clear that these are not erroneous data points — they correspond to **premium, high-end laptops** (e.g., Razer Blades, Dell Workstations with 4K screens, Intel Xeon CPUs, and high-tier GPUs). These are legitimate market entries in the luxury/professional segment. Removing them would introduce selection bias and the model would underperform on high-end predictions. So the decision was made to **retain all 28 of these observations**.

2.4 Column-Level Quality Issues

Beyond the global issues, I identified several column-specific quality and consistency problems:

- Unnamed: 0: Redundant index column with no predictive value.
- Company: 'Vero' was listed as a separate manufacturer, but its actually just a product series under Acer.
- Inches: Some values had '?' representing missing/invalid data. Others appeared to be in centimeters instead of inches (values > 18.4).
- Ram: Values like '8GB' had unit suffixes that needed stripping. One record had '1GB' which was clearly incorrect — verified via external research to be '8GB'.
- Memory: One record showed '?' indicating a missing value. Also the column was storing combined information (type + size) in a single string.
- Weight: One entry was 0.0002kg (physically impossible), another was 11.10kg (unrealistic for a consumer laptop). Some values also had 'kg' suffix and '?' characters.
- ScreenResolution: Combined resolution dimensions and panel technology in one string, making it analytically unusable without parsing.
- Cpu: Similarly, brand, model name, and clock speed were all concatenated into one field.

3. Preprocessing Steps

3.1 Handling Missing Values

As described in Section 2.1, the 30 completely empty rows were removed via listwise deletion using `df.dropna(inplace=True)`, followed by a reset of the DataFrame index. For the '?' characters found in the Inches, Memory, Weight, and ScreenType columns, these were first replaced with `np.nan` to convert them into proper NaN values. Then each was imputed using grouped imputation — where I formed clusters of similar laptops (based on Company, TypeName, ScreenResolution, CPU, RAM etc.) and imputed using the mean (for numeric) or mode (for categorical) within each group. This approach is alot more accurate then global mean/mode imputation because it accounts for the context of each laptops specifications.

3.2 Fixing Inconsistencies and Structural Issues

3.2.1 Unnamed: 0 Column

This column was simply the row index carried over from the source CSV. It was dropped immediately as it adds no meaningful information and would have unnecessarily increased dimensionality.

3.2.2 Company Column

'Vero' is not an independent laptop brand — it's a product line within Acer. All instances of 'Vero' in the Company column were replaced with 'Acer' to maintain consistency and avoid the model treating it as a completely separate manufacturer.

3.2.3 Inches Column

After handling the '?' values with NaN and imputation, I checked for outlier inch values. Any value exceeding 18.4 inches was determined to likely be in centimeters, and was converted using the formula: $\text{inches} = \text{cm} / 2.54$. The lower extreme values (like 10.1 inches) were verified to be legitimate small-form-factor laptops and retained as is.

3.2.4 RAM Column

The 'GB' suffix was stripped from all values using `.str.strip('GB')`, and the column was converted to integer datatype. One erroneous record with '1GB' RAM was corrected to '8GB' based on external verification of the product specifications.

3.2.5 Weight Column

The 'kg' suffix was removed and the column converted to float. Two physically impossible values (0.0002kg and 11.10kg) were replaced with NaN and subsequently re-imputed using grouped mean imputation (grouped by Company, TypeName, Inches, and CPU).

3.2.6 ScreenResolution Column

This column was parsed and split into two separate columns: ScreenResolution (containing only the resolution string like '1920x1080') and ScreenType (containing the panel description like 'IPS Panel Full HD', 'Touchscreen', etc.). Empty ScreenType values after the split were imputed iteratively using grouped mode imputation.

3.2.7 CPU Column

The Cpu column was decomposed into three new features: Cpu_brand (e.g., Intel, AMD), Cpu_speed (clock speed in GHz, extracted from the last token and numeric-converted), and Cpu_type (the model name in between). The original Cpu column was then dropped.

3.2.8 Memory Column

The Memory column was decomposed into three separate numeric columns: SSD_Storage, HDD_storage, and Flash_Storage, each expressing the respective capacity in GB. This was done via a custom regex-based Python function that parsed the combined strings. The original Memory column was then dropped. This approach preserves the distinction between storage types, which is important because SSD-equipped laptops tend to command a higher price than HDD-equivalents.

4. Justification for Preprocessing Decisions

4.1 Why Listwise Deletion for the 30 Empty Rows?

Imputation requires some anchor information. Since all 12 features including the target Price were null, any imputed value would have been entirely synthetic — essentially fabricating data. Retaining such rows would dilute the dataset and potentially introduce systematic bias. Listwise deletion was therefore the statistically sound choice.

4.2 Why Retain the 28 High-Price Outliers?

In price prediction models, extreme-but-valid observations carry critical information about the relationship between premium specifications and price. Removing these records would have caused the model to systematically underpredict prices for high-end laptops, because it would

have never seen examples of how premium GPUs, 4K screens, and Xeon CPUs affects price. Retaining them ensures the model learns the full price spectrum of the market.

4.3 Why Grouped Imputation Over Global Mean/Mode?

Global mean or mode imputation assumes all observations come from the same distribution, which is clearly wrong in a dataset with such diverse product categories. A 10-inch netbook and a 17-inch gaming laptop should not share the same imputed screen size. By grouping on similar specs before imputing, I ensured the imputed values was contextually appropriate and aligned with real-world product configurations.

4.4 Why Split ScreenResolution and CPU Instead of Label Encoding Them?

Both ScreenResolution and CPU contained multiple pieces of semantically distinct information packed into a single string. Simply applying label encoding would have treated 'IPS Panel Full HD 1920x1080' and 'IPS Panel 4K Ultra HD 3840x2160' as arbitrary integers with no ordinal meaning. By decomposing them, we extracted genuinely meaningful sub-features that separately captures the panel quality, resolution tier, processor brand, model, and speed — all of which individually carry predictive power for price estimation.

4.5 Why Not Extract Just the GPU Brand?

While extracting just the GPU brand (AMD, Nvidia, Intel) would simplify the column, it would have resulted in significant information loss. The difference between an Nvidia GeForce 940MX and an Nvidia RTX 3080 is enormous in terms of both performance and price impact. Reducing these to just 'Nvidia' would make both laptops appear equivalent from the models perspective, which is clearly not accurate.

4.6 Encoding Strategy Justification

A hybrid encoding approach was applied as follows:

- Ordinal Encoding for ScreenType and ScreenResolution: Both have a natural quality hierarchy (e.g., 4K > QHD > FHD > HD), and ordinal encoding preserves this ordering which is directly relevant to price.
- One-Hot Encoding for TypeName, Cpu_brand, and OpSys: These are low-cardinality nominal features with no natural ordering. One-hot encoding avoids imposing any false ordinal relationship and keeps the number of new columns manageable.
- Label Encoding for Company, Cpu_type, and Gpu: These have high cardinality (many unique values). One-hot encoding them would have resulted in a very wide, sparse feature matrix, which is undesirable. Label encoding converts them to integers without the dimensionality explosion.

4.7 Why MinMax Scaling Instead of Standardisation?

The dataset was not normally distributed (as confirmed by the right-skewed price histogram), making Z-score standardisation a poor fit. MinMax scaling was chosen because we have reasonably well-defined bounds for most features (e.g., screen size, weight), and it preserves the shape of the distribution. Additionally, to prevent data leakage, the scaler was fitted only on the training set and then applied (transform only) to the test set.

4.8 Why ProcessingPower Feature Was Created?

A new composite feature $\text{ProcessingPower} = \text{RAM} \times \text{Cpu_speed}$ was engineered to capture the interaction between memory and processing speed, which together determine overall system performance more effectively than either feature alone. Laptops with high RAM and high clock

speed will command a premium, and this combined feature helps the model capture that relationship. The original Ram and Cpu_speed columns were then dropped to avoid multicollinearity.

5. Final Dataset Summary

5.1 Cleaned Dataset Statistics

After completing the entire preprocessing pipeline, the final cleaned and machine-learning-ready dataset has the following characteristics: 1,273 rows and 27 features (including the target Price column). All features are numeric (integer or float), all values have been scaled using MinMax scaler to the [0, 1] range, and there are no null values or inconsistencies remaining.

Attribute	Value
Total Records	1,273 (after removing 30 null rows)
Total Features (after preprocessing)	27 columns (including target variable Price)
Target Variable	Price (continuous, not scaled — used as-is for regression)
Missing Values	Zero (all null values either removed or imputed)
Duplicate Records	Zero
Feature Encoding	Ordinal (ScreenType, ScreenResolution), OHE (TypeName, Cpu_brand, OpSys), Label (Company, Cpu_type, Gpu)
Feature Scaling	MinMax Scaling applied to all predictor features
Train / Test Split	80% Training (1,018 rows) / 20% Test (255 rows)
Feature Engineering	ProcessingPower = RAM × Cpu_speed (new composite feature)
New Derived Columns	Cpu_brand, Cpu_type, Cpu_speed, ScreenType, ScreenResolution, SSD_Storage, HDD_storage, Flash_Storage

5.2 Model Validation Results

To validate that the preprocessing pipeline was actually effective, two baseline regression models was trained and evaluated on the cleaned and scaled dataset:

Model	Mean Squared Error	R ² Score (Accuracy)
Linear Regression	382,517,897.42	0.7363 (~73.6%)
Random Forest Regressor	225,761,240.25	0.8444 (~84.4%)

5.3 Final Feature List

The table below lists all 27 columns present in the final cleaned dataset along with their origin:

Column Name	Origin	Notes
Company	Original (encoded)	Label encoded
Inches	Original (cleaned)	cm values converted to inches
ScreenType_Rank	Derived from ScreenResolution	Ordinal encoded (0–4)
ScreenResolution_Rank	Derived from ScreenResolution	Ordinal encoded (0–4)
Cpu_type	Derived from Cpu	Label encoded
SSD_Storage	Derived from Memory	GB capacity (integer)
HDD_storage	Derived from Memory	GB capacity (integer)
Flash_Storage	Derived from Memory	GB capacity (integer)
Gpu	Original (encoded)	Label encoded, full model retained
Weight	Original (cleaned)	Noisy values imputed, float
ProcessingPower	Engineered feature	RAM × Cpu_speed
TypeName_* (6 cols)	Original (OHE)	One-hot encoded
Cpu_brand_* (3 cols)	Derived from Cpu (OHE)	One-hot encoded
OpSys_* (7 cols)	Original (OHE)	One-hot encoded
Price	Original (Target)	Original Price of the Laptop (Untouched)

End of Report